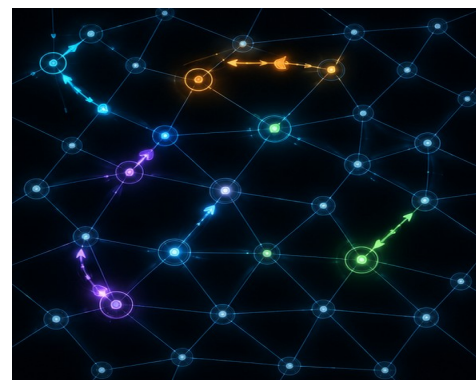




v1.0 P.G.L. Agents et Graphes

FILIERE ING1-GI • 2025-2026

AUTEURS E.ANSERMIN - R.GRIGNON

E-MAILS eva.ansermin@cyu.fr - romuald.grignon@cyu.fr

DESCRIPTION

- Le but de ce projet est de créer une application graphique qui permette de visualiser, modifier et simuler le déplacement d'agents dans un graphe. Ces agents posséderont des propriétés individuelles comme la vitesse et le mode de déplacement, la destination, ou encore la propension à être dans un environnement dense ou non.
- Le but est de pouvoir interagir visuellement sur cette carte 2D et d'observer les modifications en direct.
- Le périmètre technique est détaillé dans ce document. Le service rendu d'un point de vue utilisateur doit, quant à lui, être en adéquation avec le périmètre Ethique/Design sur lequel vous avez travaillé pendant les semaines précédentes. Ce périmètre devra être fourni à votre tuteur de projet (livrables Ethique/Design) pour qu'il puisse vérifier la cohérence.

INFORMATIONS GENERALES

- **Taille de l'équipe**
Ce projet est un travail d'équipe. Il est préférable de se réunir en groupe de 5 personnes. Si le nombre d'étudiants n'est pas un multiple de 5 et/ou si des étudiants n'arrivent pas à constituer des groupes, c'est aux chargés de projet de statuer sur la formation des groupes. Pensez donc à anticiper la constitution de vos groupes pour éviter des décisions malheureuses.
- **Démarrage du projet et jalons**
Vous obtiendrez de plus amples informations quant aux dates précises de rendu, de soutenance, les critères d'évaluation, le contenu du livrable, ..., quand le projet démarrera officiellement. Quel que soit le planning initial du projet, vous veillerez à **planifier** plusieurs **rendez-vous** d'avancement avec votre chargé(e) de projet. C'est à votre groupe de prendre cette **initiative**. L'idéal est de faire un point 1 ou 2 fois par semaine mais cette fréquence est laissée libre en fonction des besoins identifiés avec le tuteur de projet.

➤ **Versions de l'application**

Pour éviter d'avoir un projet non fonctionnel à la fin, il vous est demandé d'avoir une version en ligne de commande fonctionnelle. Ceci vous permettra de tester votre modèle de données indépendamment de l'interface graphique. C'est la version avec l'interface graphique **JavaFX** qui sera bien entendu **évaluée** mais dans le cas où certaines fonctionnalités ne seraient pas visibles avec cette interface, vous devez pouvoir présenter toutes les fonctionnalités de votre code Java en ligne de commande.

➤ **Dépôt de code**

Vous devrez déposer la **totalité** des fichiers de votre projet sur un dépôt central **Git**. Il en existe plusieurs disponibles gratuitement sur des sites web comme github.com ou gitlab.com. La fréquence des **commits** sur ce dépôt doit être au minimum de **1 commit/jour**.

➤ **Rapport du projet**

Un rapport écrit est requis, contenant une brève description de l'équipe et du sujet. Il décrira les différents problèmes rencontrés, les solutions apportées et les résultats. L'idée principale est de montrer comment l'équipe s'est organisée, et quel était le flux de travail appliqué pour atteindre les objectifs du cahier des charges. Le rapport du projet peut être rédigé en français ou en anglais. Ce rapport contiendra en plus les éléments techniques suivants : un document de conception UML (diagramme de classe) de votre application, et un document montrant les cas d'utilisations.

➤ **Démonstration**

Le jour de la présentation de votre projet, votre code sera exécuté sur vos machines. Prévoyez d'amener **2 machines** fonctionnelles. Il sera possible que le jury vous demande d'effectuer des modifications en direct de votre code, pendant que vous présentez le fonctionnel de votre application d'où la nécessité d'avoir ces 2 machines.

Si le jury vous demande de faire des modifications, il sera sûrement nécessaire de le faire sans l'utilisation de l'IA, il est donc important que vous maîtrisiez votre code ce jour là.

La version utilisée sera la **dernière** fournie sur le **dépôt** Git avant la date de rendu. Même si vous avez une nouvelle version qui corrige des erreurs ou implémente de nouvelles fonctionnalités le jour de la démonstration, c'est bien la version du rendu qui sera utilisée.

Si vous avez une version **livrée en retard** et que vous souhaitez qu'elle soit utilisée lors de la présentation, il faudra le préciser **par écrit à l'avance**, afin que nous puissions en tenir compte de ce retard dans la notation.

Le jour de la présentation, vous devrez, en **ligne de commande**, récupérer votre dépôt Git, compiler votre projet, et le lancer. Il sera **interdit** de faire le test de votre application via un environnement de développement.

➤ Organisation de l'équipe

Votre projet sera stocké sur un dépôt git (ou un outil similaire) tout au long du projet pour au moins trois raisons :

- éviter de perdre du travail pendant le développement
- être capable de travailler en équipe efficacement
- partager vos progrès de développement facilement avec votre chargé(e) de projet.

De plus il est **recommandé** de mettre en place un environnement de travail en équipe (Slack, Trello, Discord, ...).

CRITERES DE NOTATION

- Le but principal du projet est de fournir une **application fonctionnelle** pour l'utilisateur. Le programme doit correspondre à la description en début de document et implémenter toutes les fonctionnalités listées.
- Votre application devra reprendre autant que possible le **périmètre Ethique et Design** sur lesquels vous avez travaillé au préalable.
- Votre projet doit être utilisable au clavier ou à la souris en fonction des fonctionnalités nécessaires.
- Tous les éléments de votre code (variables, classes, noms de fichiers ou dossiers, fonctions, commentaires) seront écrits **en langue anglaise** obligatoirement.
- Votre code devra être **commenté** de telle manière que l'on puisse utiliser un outil de génération de documentation automatique de type **JavaDoc**. Un dossier contenant la documentation générée par vos soins sera présent dans votre dépôt de code lors de la livraison.
- Votre application ne doit jamais s'interrompre de manière intempestive (crash), ou tourner en boucle indéfiniment, quelle que soit la raison. Les exceptions qui s'affichent sur la console sont des Exceptions qui n'ont pas été gérées par votre code et sont bien considérées comme des crash (même si dans le cas d'une Application JavaFX, le processus d'affichage graphique est séparé et donc que la fenêtre reste ouverte). Une Exception non gérée sera très pénalisant sur la notation.
- Toutes les erreurs doivent être gérées correctement. Il est préférable d'avoir une application stable avec moins de fonctionnalités plutôt qu'une application complète mais qui plante trop souvent.
- Votre application devra être organisée en modules afin de ne pas avoir l'ensemble du code dans un seul et même fichier par exemple. Apportez du soin à la conception de votre projet avant de vous lancer dans le code.
- Le livrable fourni à votre tuteur de projet sera simplement l'URL de votre dépôt Git accessible publiquement à la date du rendu. **Inutile** d'envoyer des **fichiers** par **email** ou **messaging** instantanée : seuls les fichiers présents dans le dépôt à la date du dernier commit antérieur à la date limite seront pris en compte. Si vous décidez de livrer des fichiers oubliés ou modifiés après la date limite, et que vous insistez pour que cette version soit prise en compte, alors vous aurez des **pénalités de retard** qui s'appliqueront et elles seront très punitives sur la note finale. Prévoyez vos tests et votre livraison à l'avance.

- Toutes les fonctionnalités décrites dans la suite du document sont purement techniques, mais le but principal d'une application est de répondre à une problématique d'un utilisateur : il faut donc adapter ce cahier des charges pour répondre à votre périmètre Ethique/Design. De fait, les statistiques, les actions possibles, les affichages, ..., doivent être adaptées le plus possible à votre projet personnel.

DETAILS DU PROJET

➤ Interaction avec le graphe

L'utilisateur peut ajouter/modifier/déplacer/supprimer des **nœuds** et des **arêtes** dans le graphe affiché à l'écran. Cette fonctionnalité doit pouvoir se faire en direct pendant la simulation.

Une option d'ajout de masse avec des noeuds et des arêtes aléatoires doit être proposée (ajouter X nœuds)

Des **agents** peuvent être **ajoutés/supprimés** manuellement ou en masse. Ces agents disposent de propriétés qu'il doit être possible de modifier. Pour l'ajout, ces valeurs de propriétés initiales peuvent être piochées aléatoirement depuis des plages qui seront réglables dans l'application.

Si un nœud ou une arête est supprimé (la suppression d'un nœud entraîne la suppression des arêtes connectées), les agents présents dans ces zones doivent être déplacés.

Si l'agent est dans une arête, on le place dans le nœud d'où il vient. Si l'agent est dans un nœud supprimé, on le place immédiatement dans un des nœuds adjacents.

Le fait de placer de nouveaux agents dans un nœud comme ceci, va peut être faire dépasser la capacité maximale du nœud : on accepte cet état mais le nœud est en mode « forte congestion » et les agents mettent 2 cycles de simulation pour passer dans une arête disponible (jusqu'à ce que le nombre d'agents revienne à la normale).

Sur l'application, il est possible de sélectionner des nœuds ou des arêtes et de voir des statistiques à leurs sujets (nombre d'agents passés, vitesse moyenne, ...) mais également de visualiser graphiquement la densité dans le réseau (gradient de couleurs, ...).

L'utilisateur doit pouvoir sélectionner un agent pour visualiser le trajet qui lui reste à effectuer. Cela doit permettre de visualiser le comportement (changement de route si congestion, laisser passer tous les autres agents avant de s'engager, éviter les zones denses, ...)

➤ Simulation temporelle

Pour la simulation, le déplacement des agents doit se faire avec une vitesse réglable (l'intervalle de simulation). Une option de défilement pas à pas doit être disponible, ainsi que des boutons de pause et remise en route.

On part du principe que le déplacement dans un nœud n'est pas limité, voire il est instantané pour tous les agents (un nœud est un point dans l'espace). Par contre l'application devra gérer le fait que des agents veulent passer dans une arête et c'est là que des goulots d'étranglement peuvent apparaître (et donc des ralentissements) en fonction de capacités des arêtes (voir les propriétés) et du comportement des agents (voir les propriétés des agents).

A chaque intervalle de simulation, X agents peuvent entrer dans une arête : X étant le nombre d'agents qui peuvent progresser dans cette arête en même temps (pour les 2 sens confondus).

Cette fonctionnalité peut être gérée de 3 manières qu'il vous faudra choisir :

- un agent avant d'entrer dans l'arête vérifie qu'il y a de la place pour progresser quelle que soit le sens (il regarde le nœud en face pour savoir si son « couloir » de déplacement est disponible)
- l'agent entre dans l'arête sans regarder l'encroisement à l'intérieur, et risque de collisionner un autre agent qui vient dans le sens inverse. Ceci peut entraîner un blocage temporaire/définitif de l'arête.
- définir pour une arête, un nombre de « couloirs » pour chaque sens de déplacement, et la problématique de collision disparaît : ceci peut être assez limitant sur le côté comportemental des agents (un agent stressé qui fuit une position ne va pas suivre les règles de déplacements établies (tenir son couloir, ne pas double, ne pas pousser, ...))

Quand un agent atteint sa destination, il faut un critère de décision sur la suite à appliquer (choisir une nouvelle destination aléatoire, supprimer l'agent, ...).

Les propriétés présentées dans les paragraphes suivants sont à titre indicatif et ne sont pas exhaustives : il vous faut choisir celles qui ont le plus de sens en fonction de votre périmètre d'application.

➤ **Propriétés des nœuds :**

- nombre d'agents qu'il peut stocker au maximum (volume maximal)
- fermé/bloqué (travaux, ...)
- attractif / répulsif (animation, nourriture, zone de danger, ...)

➤ **Propriétés des arêtes :**

- sens de déplacement possible (certaines arêtes sont orientées)
- distance entre les nœuds (il est possible d'utiliser directement le placement spatial des nœuds, à ce moment là, les coordonnées des éléments graphiques deviennent des propriétés importantes à utiliser)
- réduction ou accélération de vitesse de déplacement des agents (débris encombrants, tapis roulants, ...)

- nombre d'agents qui peuvent être à la même position dans l'arête (largeur de l'arête) ce qui permet de gérer le fait qu'un agent double ou non un autre agent devant lui qui avance moins vite)

➤ **Propriétés des agents :**

- identifiant unique
- position (au sein d'une arête ou au niveau spatial global)
- vitesse de déplacement maximale
- comportement (laisse passer les autres, prend la priorité systématiquement, suit et reste derrière l'agent le plus proche qui va dans sa direction, ...)
- état de l'agent (calme, paniqué, folie, ...) : le comportement peut changer en fonction de l'état
- tolérance à la densité/congestion
- destination (aucune, déplacement aléatoire, fixe et cherche à se rapprocher, fixe mais cherche à s'éloigner, aléatoire mais cherche à s'éloigner, se déplace vers les zones les plus/moins denses, ...) : la destination peut changer en fonction de l'état de l'agent

➤ **Choix de trajet**

Chaque agent peut choisir un trajet en fonction de sa destination et d'autres critères comme :

- la longueur du trajet (trajet le plus court)
- le temps de trajet (prend en compte la vitesse de déplacement et la congestion dans le calcul du plus court chemin)

➤ **Import / Export**

A tout moment l'application peut sauvegarder dans un fichier binaire l'état de la simulation (agents, nœuds, arêtes) pour pouvoir la restaurer plus tard.

RESSOURCES UTILES

GitHub

- <https://www.github.com>
- <https://docs.github.com/en/get-started/quickstart/hello-world>

Patrons de conception

- https://fr.wikipedia.org/wiki/Patron_de_conception

Sérialisation

- <https://fr.wikipedia.org/wiki/Sérialisation>